Machine Learning Fundamentals in Python
-
Data Wrangling

Martin Rerabek

ZHAW

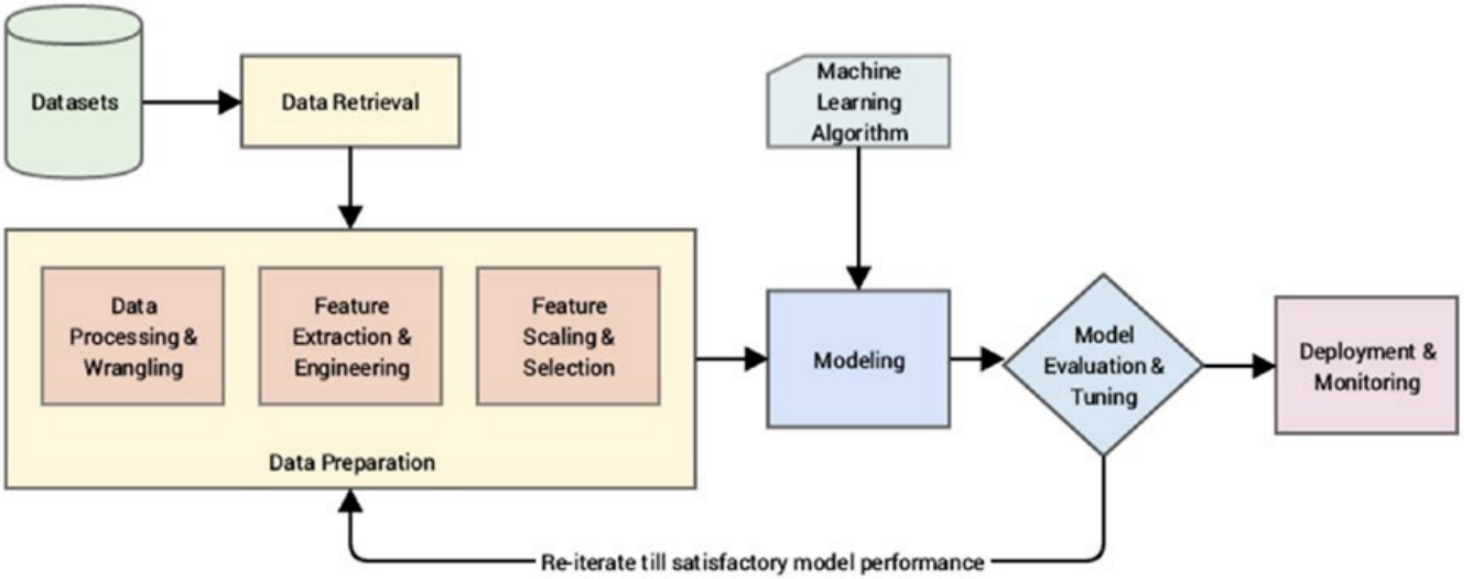
rera@zhaw.ch

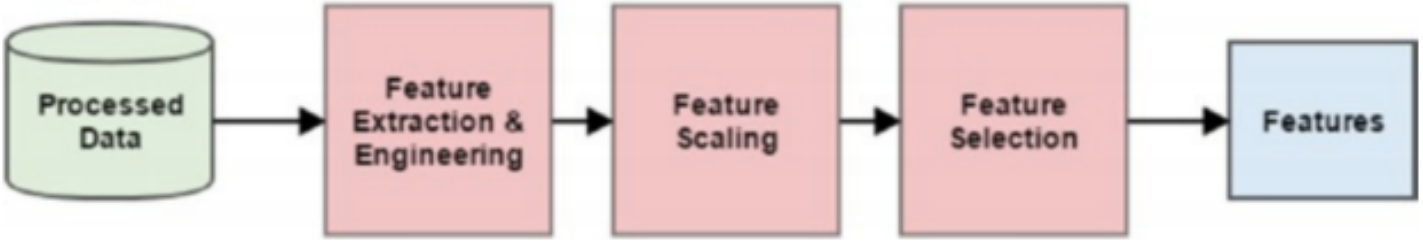
Week 1

Overview

1. ML General Pipeline
2. Data Wrangling
   - Data Collection and Analysis
3. Feature Engineering
4. Feature Scaling

ML General Pipeline





What is data wrangling?

- Transforming and mapping data from one "raw" data form into another format to make it more appropriate and valuable for a variety of purposes such as analytics
  - Collection
  - Description
  - Processing
  - Visualization

  } Analysis

  } Data understanding
- Feature Engineering/Selection
- "Coming up with features is difficult, time-consuming, requires expert knowledge. 'Applied Machine Learning' is basically feature engineering." - Prof. Andrew Ng

Data Understanding

- Collection - get data needed for your objective
  - sources - open source, archives, web, (toy datasets in Python), format of data - CSV, JSON, XML, HTML, SQL
- Description
  - initial analysis to understand more about data volume, attributes, relationships, basic descriptive statistics; num/text/cat
- Exploratory vs Data Quality Analysis
  - Explore, describe, and visualize data attributes (variables, data types)
  - Select data and attributes subsets that seem most important for the problem
  - Extensive analysis to find correlations and associations and test hypotheses
  - –
  - Missing values
  - Inconsistent values
  - Wrong information due to data errors (manual/automated)
  - Wrong metadata information
  - data imputation - many methods, nearest, mean, predictive mean matching

Data Description

- Numeric
  - Simples, often directly usable and understood by ML alg.
    - Normalization, binning, quantization, etc - based on requirements
- Text
  - Unstructured, alphanumeric
- Categorical - between numeric and text
  - Nominal - no ordering possible, e.g. color
  - Ordinal - can be ordered based on context, e.g. category of economic status low/medium/high

Data Summarization and Visualization

- Process of preparing a compact representation of raw data at hand - more in Colab
- Visualization examples using Pandas/Matplotlib

Data Summarization and Visualization

- Colab Session

What is Feature Engineering?

- process of transforming raw data into features
  - better represent the problem to the predictive model
  - resulting in improved model accuracy
- few examples
  - get person age from birth date and the current date
  - get the average and median view count of specific songs and music videos
  - get word counts from text documents
  - get pixel information from raw images
  - get occurrences of various grades obtained by students

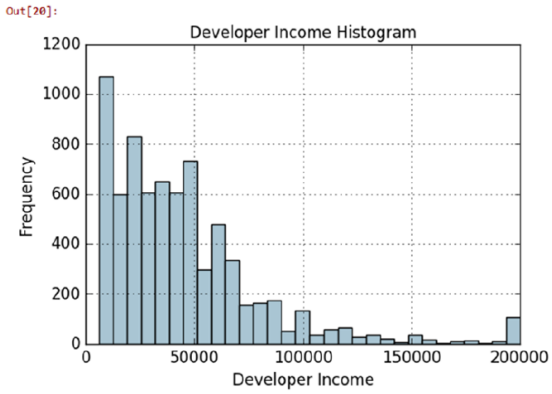
Why Feature Engineering?

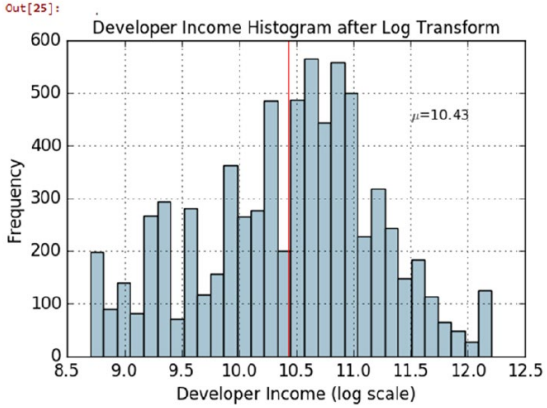
- Better representation of data
  - features vs underlying data
  - features => better understood by ML algorithm
  - easy to visualize?!
- Better performing models!
  - right feature set
  - better features => better models
- Essential for model building and evaluation
  - raw data cannot be used to build ML model
  - ML pipeline - feature set reiteration
- More flexibility on data types
  - numeric vs more complex data types
  - challenges to transform unstructured data - text, images, video
- Emphasis on the business and domain
  - raw data often can be interpreted in various ways
  - step back and try to understand the domain and the business better
  - domain expert needed often to create suitable feature set

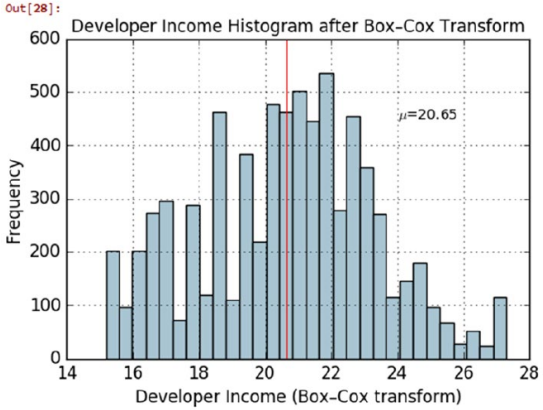
Feature Engineering on Numeric Data

- Raw measures
  - Values - specific measurement of specific variable
  - Counts - indicate counts, frequencies and occurrences of specific variable
- Binarization - in recommendation system to know if song was listened or not
- Rounding - decrease level of precision, e.g. for percentages
- Binning - e.g. age range assignment to a bin/category
  - Fixed-Width
  - Adaptive - quantile based - helps to prevent irregular and not uniform bin population
- Interactions - operations between features
- Statistical transformation
  - Log transform - useful when applied to skewed distributions
  - Box-Cox Transform - to improve distribution of data to be more normal

Statistical transformation







- Box-Cox
- $y = f(x, \lambda) = x^{\lambda} = \begin{cases} \frac{x^{\lambda}-1}{\lambda} & \text{for } \lambda > 0 \\ \log_e(x) & \text{for } \lambda = 0 \end{cases}$
  - Output y is a function of input x and transformation parameter λ such that when $\lambda = 0$, the resultant transform is the natural log transform, which we discussed earlier
  - The optimal value of λ is usually determined using a maximum likelihood or log-likelihood estimation

Feature Engineering on Categorical Data

- Transforming Nominal Features - e.g movie genre to genre label
- Transforming Ordinal Features - similar, but order matters
- Encoding
  - One Hot Encoding Scheme - consider numeric representation of any categorical feature with m labels, the one hot encoding scheme, transforms the feature into m binary features, which can only contain a value of 1 or 0, (sklearn.preprocessing import OneHotEncoder, LabelEncoder)
  - Dummy coding, Effect coding, bin-counting, feature hashing

Feature Engineering on Text Data

- Challenges of unstructured attributes of text documents
  - Dealing with the unpredictable nature of the syntax, format, and content of the documents to extract useful information
  - Transforming these textual representations into numeric representations for ML alg.
- Many techniques
- Two aspects
  - Pre-processing and normalizing text
    - Text tokenization and lower casing
    - Removing special characters
    - Contraction expansion
    - Removing stopwords
    - Correcting spellings
    - Stemming, Lemmatization
  - Feature extraction and engineering
    - Bag of word model - build vectors from text
    - The sky is blue and beautiful

	beautiful	blue	brown	dog	fox	jumps	lazy	love	quick	sky	today
0	1	1	0	0	0	0	0	0	0	1	0

Feature Engineering on Temporal Data and Image Data

- Temporal data changing over a period of time and time-based attributes are of paramount importance in it
  - Timestamp index
  - Date-Based Features
  - Time-Based Features
- Image data features - edges, boxes, filters, usually DL

Feature Scaling

- Raw values might make models biased - toward features with high values
- Linear or logistic regression (biased) vs tree based decision models (not biased)
- recommendation to normalize and scale down features
- Standardized Scaling $\approx$ Z-norm Scalling
  - removing the mean and scaling the variance to be 1
  - $feature_{scaled}(x_i) = \frac{x_i - \mu_i}{\sigma_i}$
- Min-Max Scaling
  - output in range of [0, 1] or arbitrary range - be aware of outliers
  - $feature_{scaled}(x_i) = \frac{x_i - \min(x)}{\max(x) - \min(x)}$
- Robust Scaling
  - remove outlier disadvantages using IRQ range between 1 and 3 quartile
  - $feature_{scaled}(x_i) = \frac{x_i - median(x)}{IRQ_{1,3}(x)}$

Feature Selection

- Engineering features helps to capture some latent representations and patterns in the underlying data
- Dealing with feature sets of thousands of features - not good idea
- curse of dimensionality - more features means more complex model difficult to interpret, specialized model = over-fitting
- The ultimate objective - select an optimal number of features to train and build models that generalize very well on the data and prevent overfitting
- Feature selection strategies
  - Filter methods - select features purely based on metrics like correlation, mutual information - check each feature vs variable to be predicted
  - Wrapper methods - recursive approach to build multiple models using feature subsets and select the best subset
  - Embedded methods - combine the above two by using ML to rank features - random forest